
AlphaEvolve-Style Program Optimization for Deterministic Chart VLM Inference

Yuzhou Zhu

1730694701@mail.dlut.edu.cn

Abstract

We study program-level optimization of a fixed vision-language inference pipeline for chart understanding under strict reproducibility constraints. Following the project setting, we keep the model family fixed to Qwen3-VL-2B-Instruct and Qwen3-VL-2B-Thinking, and optimize only the inference program rather than training a new model. Our development benchmark is a local 128-sample descriptive subset derived from CharXiv, evaluated by exact lowercase string match. Starting from a naive baseline, we combine manual engineering with an AlphaEvolve-style code evolution system that performs direct code mutation, candidate validation, population management, elite retention, and Pareto tracking. On this local development harness, the starting baseline achieves 38.28% accuracy with 1.49 seconds per query. Manual optimization improves performance to 63.28% for the Instruct model and 55.47% for the Thinking model, while evolutionary program search further improves accuracy to 66.41% and 57.81%, respectively. The fastest final system is the manual Instruct variant at 0.382 seconds per query, while the best overall system is the evolved Instruct variant under the combined objective score = accuracy - $0.05 \times \text{avg_time_per_query}$. These results suggest that, for this exact-match chart question answering setting, deterministic output control and conservative suppression of false positives matter more than complex architectural changes.

1 Introduction

Chart understanding is a practical and challenging multimodal task because useful answers often depend on visually grounded extraction of titles, axis labels, tick values, legends, colorbars, trends, and subplot structure from naturally occurring scientific figures. CharXiv was introduced to make chart understanding more realistic by collecting diverse charts from scientific papers rather than relying primarily on simplified synthetic settings [1]. In parallel, AlphaEvolve demonstrated that large language models can improve programs through an evolutionary loop that mutates code, evaluates candidates, and retains promising solutions over time [2]. These two ingredients motivate the present study: instead of training a new chart model, we ask whether a fixed small vision-language model can be made substantially better through inference-program optimization alone.

Our task is intentionally constrained. We do not change the required model family, do not fine-tune weights, and do not alter the benchmark into a new task. Instead, we optimize a fixed `vlm_inference(image_path, question)` entry point under greedy decoding only. This makes the problem closer to systems-oriented inference engineering than to conventional model training. It also makes reproducibility especially important, because improvements must arise from better prompting, stronger output control, more effective deterministic post-processing, and better search over program variants, rather than from stochastic training effects.

A central challenge in this project is that the local development harness uses exact lowercase string match. Under such an evaluator, many failures are not caused by complete visual misunderstanding,

but by avoidable output mismatches: verbose answers, leaked reasoning, incorrect answer prefixes, confusion between nearby text elements, or unnecessary guessing when the correct response should be `Not Applicable`. This observation guided both our manual optimization and our evolutionary search. Rather than seeking more elaborate reasoning traces, we focused on making the inference program more deterministic, more conservative, and more tightly aligned with the answer format expected by the evaluator.

Based on this setup, we make three contributions. First, we design manual inference pipelines for Qwen3-VL-2B-Instruct and Qwen3-VL-2B-Thinking that use question-type-aware prompting, strict answer extraction, and deterministic normalization to substantially improve both accuracy and latency over the naive baseline. Second, we implement an AlphaEvolve-style program optimization system that mutates code directly, validates candidate programs, tracks elites and a Pareto frontier, and searches for improved inference behavior under a joint accuracy-speed objective. Third, we provide an empirical comparison across baseline, manual, and evolved variants on the local CharXiv development harness, including negative results, search stochasticity, and the final distinction between the fastest submitted system and the best overall submitted system.

2 Task Setup and Evaluation Protocol

2.1 Local Development Benchmark

Our experiments are conducted on a local development harness derived from CharXiv [1]. Importantly, this is not the full official benchmark evaluation. The local harness uses the first 128 samples from `charxiv/data/descriptive_val.json` and scores predictions using exact lowercase string match. There is no GPT-based grading in this development pipeline. As a result, the numbers reported in this paper should be interpreted as local development results rather than official held-out benchmark scores. This distinction is important because output formatting and deterministic answer normalization play an unusually large role under exact-match evaluation.

2.2 Models and Inference Constraints

We follow the project requirements and keep the model family fixed to Qwen/Qwen3-VL-2B-Instruct and Qwen/Qwen3-VL-2B-Thinking. No quantization is used in the final submitted inference files. All final inference entry points process one chart-question pair at a time, so the effective batch size is 1. Greedy decoding is used throughout all development runs and all submitted scripts in order to preserve strict reproducibility.

2.3 Runtime Environment

The final verified environment is a single NVIDIA GeForce RTX 4090 Laptop GPU with CUDA 12.8, PyTorch 2.10.0+cu128, Transformers 4.57.1, and a Conda environment named `v1m`. We report average wall-clock time per query under this environment. Because all final entry points use single-example inference, these latency numbers should be interpreted as per-query end-to-end inference time in the submitted implementation rather than throughput under batched serving.

2.4 Metrics

We report three quantities. The first is exact-match accuracy on the 128-sample local harness. The second is average inference time per query in seconds. The third is a combined score used during evolutionary search:

$$\text{score} = \text{accuracy} - 0.05 \times \text{avg_time_per_query}. \quad (1)$$

This score favors more accurate candidates while still penalizing slower inference. It is used as the main scalar objective for candidate selection in the evolution loop, while Pareto tracking is maintained separately to preserve the accuracy-speed trade-off frontier.

3 Method

3.1 Baseline inference pipeline

The starting point of our study is the naive inference program provided in `starting_scripts.py`. This baseline is intentionally simple and serves as a reference point for both manual optimization and evolutionary search. Its behavior can be summarized by four main characteristics. First, it performs single-example inference only, processing one chart-question pair at a time. Second, it uses a generic chat-style prompt with no explicit question-type routing. Third, it applies no deterministic post-processing or answer extraction beyond returning the model output. Fourth, it uses greedy generation with `max_new_tokens=128`, but without task-specific constraints for chart elements such as legends, colorbars, subplot layouts, or the special response `Not Applicable`.

This design makes the baseline easy to run, but also exposes several predictable failure modes on the local exact-match harness. Because the prompt is generic, the model may answer in a verbose conversational style, add explanation prefixes such as “Answer:”, or include extra reasoning that is irrelevant under exact-string scoring. In addition, the absence of task-specific handling increases confusion between nearby visual elements, for example between legends and captions, or between colorbars and axis ranges. These observations motivated our decision to treat inference optimization as a program-design problem rather than merely a prompt-tuning problem.

3.2 Manual optimization for Qwen3-VL-2B-Instruct

Our manual optimization for Qwen3-VL-2B-Instruct is built around a simple hypothesis: on this local benchmark, a large fraction of the errors arise from formatting mismatch and avoidable false positives rather than from complete failure of visual understanding. Accordingly, the goal of `manual_instruct.py` is to make the inference program deterministic, concise, and conservative.

The first design change is a strict answer-only prompting strategy. Instead of allowing free-form chat responses, we instruct the model to produce one short final answer line with no explanation and no answer prefix. This directly reduces exact-match failures caused by surplus text. The second design change is question-type-aware prompting. Before generation, the program classifies the input question into common chart-reading categories, such as title, axis labels, tick values, tick differences, line counts, line intersections, legend count, legend names, colorbar maximum, colorbar range, trend, total tick count, subplot layout, and subplot count. The prompt then injects targeted task hints for the detected category, which helps the model focus on the relevant chart element rather than nearby distractors.

The third design change is deterministic post-processing. After generation, the program normalizes the raw answer into the exact form expected by the evaluator whenever such normalization can be done safely. This includes preserving numeric forms such as 10^{-6} and 1.00, removing unnecessary prefixes, and standardizing short categorical outputs. The fourth design change is conservative handling of missing or unclear chart elements. In particular, the program is biased toward returning `Not Applicable` when the requested element does not exist or is too ambiguous to answer confidently. This behavior is especially important for preventing false positives in legend, colorbar, and subplot-related questions.

Taken together, these changes transform the Instruct pipeline from a generic conversational interface into a deterministic chart question answering program. The guiding principle is not to make the model more expressive, but to make its outputs more evaluation-aligned. This is particularly effective in an exact-match setting, where a short correct answer is strictly better than a longer partially correct explanation.

3.3 Manual optimization for Qwen3-VL-2B-Thinking

Our manual optimization for Qwen3-VL-2B-Thinking follows the same general philosophy as the Instruct version, but it addresses a different dominant failure mode. In local development, the Thinking model tended to over-generate: even when it identified the correct answer, it was more likely to emit visible reasoning text, intermediate thoughts, or formatting that hurt exact-match accuracy. Therefore, `manual_thinking.py` focuses primarily on constraining answer exposure while preserving the same chart-specific guidance used in the Instruct pipeline.

The core mechanism is a strict tagged-answer contract. The system prompt instructs the model to reason internally but output only a final answer enclosed in `<answer>...</answer>`. In addition, the visible `<think>` handoff is pre-closed before generation so that the output channel is biased away from exposed reasoning. After generation, the program extracts only the content inside the answer tags and discards all other text. This extraction step is deterministic and ensures that even if the model leaks extra content, only the final tagged answer is passed to the evaluator.

Beyond answer extraction, `manual_thinking.py` reuses the same question-type-aware prompting and deterministic normalization strategy as `manual_instruct.py`. The same categories are handled explicitly, and the same conservative policy is applied when the correct response should be `Not Applicable`. In this sense, the Thinking pipeline is not a separate method family; rather, it is a stricter output-control wrapper around the same task-aware chart inference logic.

This design reflects an important empirical finding from the project: for the Thinking model, the main challenge on the local harness is not lack of reasoning capacity, but failure to expose the final answer in a compact evaluator-friendly form. As a result, strong answer extraction and output suppression contribute more than encouraging longer chains of visible reasoning.

3.4 AlphaEvolve-style program evolution

To move beyond manual optimization, we implemented an AlphaEvolve-style program search framework for both model variants, provided as `evolve_instruct.py` and `evolve_thinking.py`. The purpose of this system is to evolve the inference program itself rather than only tune a prompt string or scalar hyperparameter. In both cases, the search starts from the best manually engineered seed and iteratively proposes program edits, validates candidates, evaluates them on the local benchmark, and retains the most promising variants.

The mutation operator is a local text mutator based on cached Qwen/Qwen3-VL-2B-Instruct. Candidate edits are proposed in compact XML `find/replace` form so that the system can directly modify source code at the program level. We use two mutation backends. The first, denoted `local_qwen`, relies only on the local cached Qwen-based mutator. The second, denoted `hybrid`, combines the same local mutator with a heuristic mutation bank that serves as a fallback when the LLM-proposed edit is invalid, unparsable, or otherwise unusable. No external API-based mutation backend is used in the final submitted system.

The search loop includes several pieces of bookkeeping to keep the search efficient and reproducible. Each candidate program is deduplicated by code hash before evaluation. Candidate code is validated before runtime execution so that syntactically broken or structurally invalid variants do not waste evaluation budget. All candidates are archived as modules, and each run records prompts, mutations, metrics, per-generation summaries, and final population snapshots. During selection, the system maintains both explicit elites and a Pareto frontier over accuracy and latency. This design is useful because the single scalar objective does not fully capture the trade-off between being slightly more accurate and being slightly faster.

For candidate selection, both evolution systems optimize the combined score

$$\text{score} = \text{accuracy} - 0.05 \times \text{avg_time_per_query}. \quad (2)$$

This objective encourages search toward higher exact-match accuracy while still penalizing slower programs. In practice, it helped retain candidates that produced meaningful accuracy gains without allowing latency to degrade excessively. At the same time, Pareto tracking preserved speed-leaning candidates that were not optimal under the scalar score alone.

The actual search configurations differed across runs. We first performed a small proxy Instruct run using 8 samples, 2 generations, 1 candidate per generation, a population size of 4, an elite size of 2, and the `local_qwen` backend. We then ran a full Instruct search on all 128 local samples using 4 generations, 5 candidates per generation, a population size of 8, an elite size of 3, and the `hybrid` backend. For the Thinking model, the full search used 128 samples, 2 generations, 4 candidates per generation, a population size of 6, an elite size of 2, and the `hybrid` backend. These settings reflect a practical compute-budget trade-off: the search had to be expressive enough to discover nontrivial program improvements, but still lightweight enough to run on a single local machine.

Empirically, the most useful evolved mutations were not dramatic architectural rewrites. Instead, they mainly strengthened conservative false-positive suppression, especially around `Not Applicable`,

Table 1: Main comparison on the local 128-sample CharXiv descriptive development harness. All optimized variants improve substantially over the starting baseline. The evolved Instruct variant achieves the best accuracy and best overall score, while the manual Instruct variant remains slightly faster.

Variant	Accuracy	Avg/query (s)	Score
Starting baseline	0.3828125	1.4922516774386168	0.3081999161280692
Manual Instruct	0.6328125	0.38159692101180553	0.6137326539494097
Manual Thinking	0.5546875	0.570147329941392	0.5261801335029304
Evolved Instruct	0.6640625	0.3864690978080034	0.6447390451095998
Evolved Thinking	0.578125	0.5762407723814249	0.5493129613809288

Table 2: Improvement summary relative to the starting baseline. All optimized systems improve both exact-match accuracy and runtime efficiency.

Variant	Accuracy gain vs. baseline	Speedup vs. baseline
Manual Instruct	+0.25	$\sim 3.91\times$
Manual Thinking	+0.171875	$\sim 2.62\times$
Evolved Instruct	+0.28125	$\sim 3.86\times$
Evolved Thinking	+0.1953125	$\sim 2.59\times$

legends, colorbars, and non-line plots. For the Instruct model, the best evolved candidate reinforced the global Not Applicable rule and tightened rejection of spurious nearby-text matches. For the Thinking model, the strongest mutations improved task hints for line-related questions on non-line plots and made legend and colorbar filtering more conservative. These outcomes are consistent with our broader finding that, on this local exact-match benchmark, disciplined output control and error-aware task logic matter more than adding complexity for its own sake.

4 Experimental Results

4.1 Main comparison across variants

Table 1 summarizes the main comparison across the starting baseline, the two manually optimized systems, and the two evolved systems. Several trends are immediately clear. First, all four optimized variants substantially outperform the starting baseline in both exact-match accuracy and average inference time per query. Second, the Instruct variants consistently outperform the corresponding Thinking variants on this local development harness. Third, the best overall result is achieved by the evolved Instruct system, while the fastest final system is the manual Instruct system.

More concretely, the starting baseline achieves an accuracy of 0.3828125 with an average runtime of 1.4922516774386168 seconds per query, corresponding to a combined score of 0.3081999161280692. Manual optimization raises the Instruct model to 0.6328125 accuracy while reducing runtime to 0.38159692101180553 seconds per query, and raises the Thinking model to 0.5546875 accuracy with 0.570147329941392 seconds per query. Evolutionary search further improves both model families. The evolved Instruct variant reaches the highest accuracy of 0.6640625 with 0.3864690978080034 seconds per query and the highest combined score of 0.6447390451095998. The evolved Thinking variant improves over its manual seed as well, reaching 0.578125 accuracy with 0.5762407723814249 seconds per query.

These results support two central conclusions. First, deterministic output control and task-aware program logic provide large gains over the naive baseline. Second, AlphaEvolve-style program search remains useful even after strong manual engineering, producing additional absolute accuracy gains of 0.03125 for the Instruct model and 0.0234375 for the Thinking model.

For completeness, Table 2 reports improvements relative to the starting baseline. Manual Instruct improves accuracy by +0.25 while running about $3.91\times$ faster. Evolved Instruct improves accuracy by +0.28125 while remaining about $3.86\times$ faster than the baseline. The Thinking variants also show clear gains, though their final accuracy and latency remain behind the corresponding Instruct systems.

To visualize the joint accuracy-latency trade-off more directly, Figure 1 plots all five evaluated variants in the accuracy–latency plane.

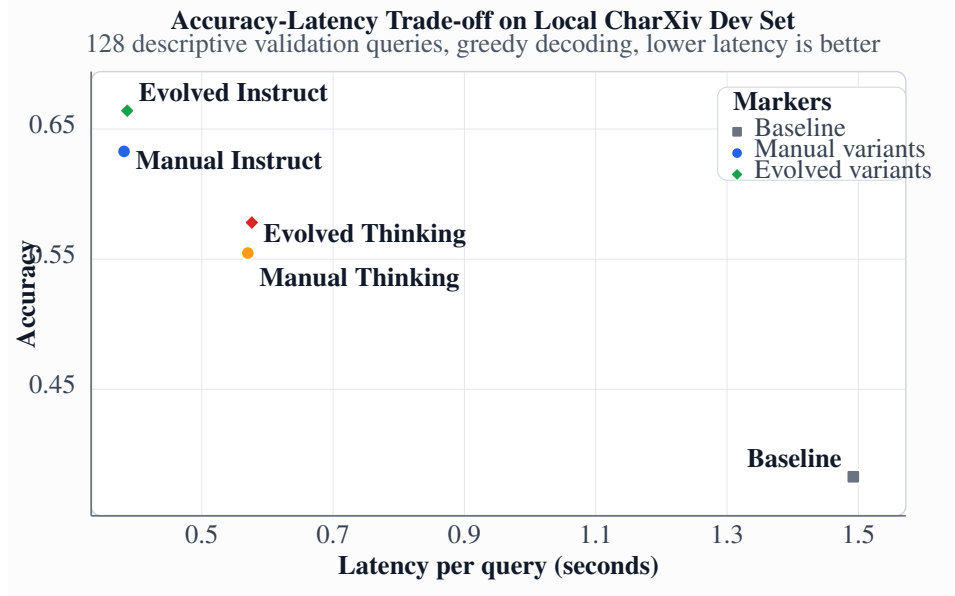


Figure 1: Accuracy–latency trade-off of the five evaluated variants on the local 128-sample CharXiv descriptive development harness. All optimized variants dominate the starting baseline. The evolved Instruct variant achieves the best overall accuracy–latency balance, while the manual Instruct variant remains the fastest verified final system.

Table 3: Final submission mapping after fresh-process verification. The fastest system and the best overall system are different files.

Submission file	Actual implementation	Selection criterion
<code>best_accuracy.py</code>	<code>evolved_instruct.py</code>	Highest accuracy
<code>best_speed.py</code>	<code>manual_instruct.py</code>	Fastest verified runtime
<code>best_overall.py</code>	<code>evolved_instruct.py</code>	Highest combined score

The figure makes two patterns especially clear: all optimized variants dominate the starting baseline, and the evolved Instruct and manual Instruct systems occupy different but competitive parts of the final trade-off frontier.

4.2 Final submission mapping

The project requires three final submission entry points beyond the manual and evolved model-specific files: `best_accuracy.py`, `best_speed.py`, and `best_overall.py`. We selected these files according to the final fresh-process verification results rather than according to intermediate search logs alone.

As shown in Table 3, `best_accuracy.py` and `best_overall.py` both map to `evolved_instruct.py`, since the evolved Instruct variant achieves both the highest exact-match accuracy and the best combined score. In contrast, `best_speed.py` maps to `manual_instruct.py`, because the final fresh-process rerun showed that the manual Instruct implementation is slightly faster than the evolved Instruct implementation. This distinction is important: the most accurate system and the fastest system are not the same submitted file.

This mapping also clarifies the practical outcome of the project. If the goal is maximum local exact-match performance, the evolved Instruct variant is the strongest choice. If the goal is minimum latency among the final verified artifacts, the manual Instruct variant is preferred. Thus, the final submission exposes both ends of the observed accuracy-speed trade-off.

Table 4: Representative evolution search outcomes from the canonical full runs. The Instruct search produced both the best final candidate and a distinct speed-leaning Pareto candidate.

Run	Candidate ID	Accuracy	Avg/query (s)
Instruct full best	g00_c005	0.6640625	0.40914330817759037
Instruct Pareto speed-leaning	g02_c015	0.65625	0.3535063248127699
Thinking full best	g01_c008	0.578125	0.607107363641262

4.3 Evolution search outcomes

Beyond the final submission files, the search logs provide a more detailed view of what the AlphaEvolve-style program optimization system actually discovered. For the Instruct model, the canonical full search run is `evolution_runs/instruct_day6_full`. Its best candidate is `g00_c005`, which achieved a search-time accuracy of 0.6640625 and an average runtime of 0.40914330817759037 seconds per query. The main mutation associated with this candidate was a strengthened global `Not Applicable` rule, which made the program more conservative when relevant chart elements were absent or ambiguous.

The same Instruct run also produced an informative speed-leaning Pareto candidate, `g02_c015`, with accuracy 0.65625 and runtime 0.3535063248127699 seconds per query. Its main mutation simplified no-prefix answer behavior, illustrating that the search did not only chase the scalar objective but also preserved candidates that offered a different balance between latency and accuracy. This is precisely why Pareto tracking was valuable in addition to scalar score optimization.

For the Thinking model, the canonical full search run is `evolution_runs/thinking_day7_full`. Its best candidate is `g01_c008`, which achieved a search-time accuracy of 0.578125 and an average runtime of 0.607107363641262 seconds per query. The main mutation for this candidate tightened line-question guidance for non-line plot types, reducing a recurring source of false positives in the Thinking pipeline.

These search outcomes reinforce two observations. First, the evolutionary system discovered useful improvements beyond strong hand-designed seeds, especially by making the inference logic more conservative and more aligned with exact-match evaluation. Second, the most effective mutations were modest but targeted: they refined answer formatting, rejection behavior, and chart-element discrimination rather than introducing sweeping structural changes. In that sense, the search validated the central design thesis of this project: on this benchmark, disciplined control of error modes is more valuable than additional complexity.

For clarity, Table 4 summarizes the most important search artifacts discussed above.

5 Analysis

5.1 Why manual optimization helped

The strong gains from manual optimization can be explained by the interaction between the local evaluation protocol and the dominant error modes of the baseline pipeline. Because the local harness uses exact lowercase string match, many failures are caused not by severe visual misinterpretation, but by outputs that are semantically plausible yet evaluator-incompatible. In practice, these include verbose responses, answer prefixes, leaked reasoning text, incorrect formatting of short categorical answers, and failure to return `Not Applicable` when the requested chart element is absent. Under such conditions, reducing output entropy is often more valuable than increasing expressive reasoning.

This is why the manual pipelines improved so substantially over the starting baseline. Both `manual_instruct.py` and `manual_thinking.py` convert the problem from open-ended multi-modal dialogue into a constrained chart question answering task with a narrow answer channel. The answer-only prompting strategy removes a large class of avoidable formatting errors. Question-type-aware hints reduce confusion among visually adjacent elements, such as legends versus captions or colorbars versus axis ranges. Deterministic post-processing further aligns outputs with the exact string forms expected by the evaluator. Finally, the conservative `Not Applicable` rule suppresses false positives when the requested chart element is missing, ambiguous, or irrelevant.

The resulting improvements are large in both accuracy and runtime. This dual gain is important. The manual systems do not merely trade extra logic for better correctness; they also reduce average time per query relative to the starting baseline. A plausible reason is that shorter, more constrained outputs reduce unnecessary generation and post hoc cleanup. Thus, the manual stage succeeds not because it adds complexity indiscriminately, but because it removes unhelpful freedom from the inference pipeline.

5.2 Why evolution still improved strong seeds

A natural question is why evolutionary search remained beneficial after the manual seeds were already strong. The answer is that the manual pipelines captured the main design principles, but they did not exhaust the space of small, interaction-heavy program edits that matter under exact-match evaluation. Once the search starts from a high-quality seed, the role of AlphaEvolve-style optimization is not to discover a completely different method family. Instead, it searches for narrow but consequential refinements that are difficult to enumerate systematically by hand.

The search outcomes support this interpretation. The best evolved candidates did not introduce dramatic architectural rewrites, novel modules, or new external tools. Instead, they repeatedly improved the same family of behaviors: stronger `Not Applicable` decisions, more conservative legend and colorbar filtering, tighter handling of line-related questions on non-line plots, and cleaner answer-prefix suppression. These are precisely the kinds of local edits that can be individually modest yet collectively important. In an exact-match setting, a small change in rejection behavior can flip multiple examples from wrong to right without any visible change in overall system structure.

The observed gains are therefore meaningful even though they are smaller than the manual-stage gains. The evolved Instruct system improves from 0.6328125 to 0.6640625, and the evolved Thinking system improves from 0.5546875 to 0.578125. These are not marginal relative improvements obtained by relaxing constraints or sacrificing reproducibility. They are achieved under the same fixed model family, the same batch size, and the same greedy-decoding regime. In this sense, the search validates the idea that program-level inference optimization remains useful even after careful manual engineering.

At the same time, the search results also show the limits of evolution in this project setting. The Day 5 proxy Instruct run did not beat its seed, and later audit reruns did not surpass the strongest earlier discovered frozen artifacts. This indicates that the search is genuinely stochastic rather than artificially designed to always improve. That negative evidence increases the credibility of the positive results: the final evolved files were chosen because they worked best under verified reruns, not because the search loop was assumed to be uniformly successful.

5.3 Instruct versus Thinking

Across all stages of the project, the Instruct variants consistently outperform the corresponding Thinking variants on the local development harness. This pattern appears in both the manual and evolved settings: `manual_instruct.py` outperforms `manual_thinking.py`, and `evolved_instruct.py` outperforms `evolved_thinking.py`. The difference is not large enough to imply that the Thinking model is unusable, but it is stable enough to suggest that the two model interfaces interact differently with this evaluation regime.

Our results suggest that the key issue is not raw reasoning ability, but answer controllability. The Thinking model’s dominant local failure mode is over-generation. Even when it identifies the correct answer internally, it is more likely to emit visible reasoning, intermediate text, or other output that is harmful under exact-match scoring. The manual Thinking pipeline partially compensates for this through a strict `<answer>...</answer>` contract, pre-closed visible `<think>` handoff, and deterministic answer extraction. These mechanisms clearly help, as shown by the substantial improvement over the baseline. However, they do not entirely erase the difference between the Thinking and Instruct variants.

By contrast, the Instruct model appears naturally better matched to a deterministic answer-only workflow. Once the prompt is constrained and the post-processing is made task-aware, the model more readily produces compact evaluator-aligned outputs. This makes it easier for both manual engineering and evolutionary search to improve the system. The final empirical picture is therefore

consistent: on this local exact-match chart QA harness, the simpler and more directly controllable Instruct interface is a better substrate for program-level optimization than the Thinking interface.

Importantly, this conclusion should be interpreted narrowly. We do not claim that Instruct models are generally superior to Thinking models for chart understanding. Rather, within this specific project setup, exact-match scoring and strict reproducibility favor output discipline more than visible reasoning depth. Under those conditions, the Instruct pathway is the more effective one.

5.4 Accuracy-speed tradeoff

A noteworthy outcome of the project is that the most accurate system is not the fastest system. The evolved Instruct variant achieves the highest exact-match accuracy and the best overall score, but the manual Instruct variant is slightly faster in the final fresh-process verification. This is why the final submission uses different files for `best_speed.py` and `best_overall.py`. The distinction is not cosmetic; it reflects a genuine trade-off discovered during the search process.

The scalar objective used in evolution,

$$\text{score} = \text{accuracy} - 0.05 \times \text{avg_time_per_query},$$

was useful for ranking candidates, but it did not eliminate the need to inspect the Pareto frontier explicitly. Some candidates achieved slightly lower accuracy with meaningfully lower latency, while others preserved or improved accuracy at a small runtime cost. The speed-leaning Instruct Pareto candidate illustrates this clearly: it was not the final overall winner, but it revealed that answer-prefix simplification and related edits could produce a different part of the trade-off curve. Maintaining both elites and a Pareto frontier was therefore an important design choice, because it preserved candidate diversity that a purely scalar selection strategy might have discarded.

From a broader perspective, the trade-off pattern also clarifies what the project optimized most effectively. The largest absolute accuracy gains came from making the system more conservative and more tightly aligned with the evaluator. Some of those changes slightly increased control overhead or reduced the aggressiveness of latency-oriented simplifications. As a result, the final best-overall system is not the same as the absolute fastest one. This is a desirable outcome rather than a flaw: it shows that the search did not collapse all objectives into a single misleading headline number.

Overall, the accuracy-speed relationship observed here is consistent with the main thesis of the paper. In this project, high performance does not come from scaling the model or increasing generation depth. It comes from disciplined program design that carefully manages output format, false-positive suppression, and task-specific answer logic. Once those mechanisms are in place, the remaining trade-off is relatively small and interpretable, which is precisely what one would want from a reproducible inference optimization pipeline.

6 Limitations

Our results should be interpreted within the constraints of the local development setting. First, the reported numbers are measured on a 128-sample local descriptive subset rather than on the full official CharXiv evaluation. Although this harness is useful for fast iteration and controlled comparison, it is relatively small, and the risk of overfitting to its specific answer distributions and formatting patterns cannot be ruled out. In particular, some of the strongest improvements in this project arise from deterministic output control and exact-match-oriented normalization, which may transfer imperfectly to evaluation settings that reward semantic correctness more flexibly.

Second, the local evaluator uses exact lowercase string match rather than a more permissive semantic judgment. This means that the system is optimized not only for chart understanding, but also for answer-format compatibility. That design is appropriate for the present project because the development protocol is fixed, but it also narrows the interpretation of the gains. Some improvements may primarily reduce formatting errors rather than substantially increase underlying visual reasoning ability.

Third, the evolutionary component is genuinely stochastic and not uniformly successful. The Day 5 proxy Instruct run did not outperform its seed, and later audit reruns did not surpass the strongest earlier frozen artifacts. This behavior is informative rather than problematic, but it does mean that

search quality depends on budget, initialization, and mutation trajectory. In a larger study, one would ideally run more seeds and report broader search variance.

Fourth, the best evolved variants were not the fastest final variants. The project therefore does not produce a single universally best system under every criterion. Instead, it yields a small frontier of useful solutions, with `manual_instruct.py` as the speed winner and `evolved_instruct.py` as the accuracy and overall-score winner. This is an honest outcome, but it also shows that the chosen scalar objective does not eliminate the underlying multi-objective nature of the problem.

Finally, we do not claim that the relative ordering observed here will necessarily hold on the hidden official test set or under alternative hardware and serving conditions. The conclusions of this paper are strongest as statements about reproducible local development on a fixed small-model chart QA pipeline. They should not be overstated as evidence of benchmark-wide gains without additional held-out validation.

7 Conclusion

We presented a study of program-level optimization for deterministic chart VLM inference under strict reproducibility constraints. Rather than changing the model family or training new weights, we optimized a fixed inference pipeline built on Qwen3-VL-2B-Instruct and Qwen3-VL-2B-Thinking. On a local 128-sample CharXiv descriptive development harness, both manual engineering and AlphaEvolve-style program search delivered substantial improvements over the naive baseline. The strongest final system, `evolved_instruct.py`, achieved 0.6640625 accuracy with 0.3864690978080034 seconds per query and the best combined score, while `manual_instruct.py` remained the fastest verified final system.

The central empirical lesson is that, in this exact-match chart question answering setting, disciplined output control matters more than added complexity. The largest gains came from answer-only prompting, question-type-aware hints, deterministic normalization, strict answer extraction, and conservative suppression of false positives such as incorrect legend and colorbar matches. Evolutionary search remained useful even after strong manual optimization, but its most successful mutations were small and targeted rather than architecturally dramatic.

More broadly, this project suggests that small multimodal models can often be improved meaningfully through careful inference-program design alone. When the task, model family, and decoding policy are fixed, program search becomes a practical way to refine behavior without sacrificing reproducibility. In that sense, AlphaEvolve-style inference optimization is a promising systems methodology: it does not replace manual insight, but it can extend and sharpen it.

References

- [1] Zirui Wang, Mengzhou Xia, Luxi He, Howard Chen, Yitao Liu, Richard Zhu, Kaiqu Liang, Xindi Wu, Haotian Liu, Sadhika Malladi, Alexis Chevalier, Sanjeev Arora, and Danqi Chen. (2024) CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. arXiv preprint arXiv:2406.18521.
- [2] Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. (2025) AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv preprint arXiv:2506.13131.

A Additional Implementation Details

A.1 Submitted files and reproducibility

The final code package contains the following required files:

- `manual_instruct.py`
- `manual_thinking.py`
- `evolved_instruct.py`
- `evolved_thinking.py`

- `evolve_instruct.py`
- `evolve_thinking.py`
- `best_accuracy.py`
- `best_speed.py`
- `best_overall.py`
- `evaluate.py`
- `reproduce.sh`
- `reproduce.py`

The final entry-point mapping used in the submission is:

- `best_accuracy.py` = `evolved_instruct.py`
- `best_speed.py` = `manual_instruct.py`
- `best_overall.py` = `evolved_instruct.py`

The reproducibility workflow is designed to re-run the key reported numbers from clean processes. The primary command is:

```
bash reproduce.sh
```

On the development machine, this was validated under Windows through Git Bash via:

```
& 'C:\Program Files\Git\bin\bash.exe' reproduce.sh
```

A.2 Manual inference implementation details

The two manual inference files preserve the same required public interface,

```
vlm_inference(image_path, question)
```

while changing the internal prompting and post-processing logic.

For `manual_instruct.py`, the main implementation choices are:

- a one-line answer-only output contract,
- question-type-aware routing for common chart-reading tasks,
- deterministic answer normalization for numeric and categorical forms,
- conservative fallback to `Not Applicable` when the requested chart element is absent or unclear.

For `manual_thinking.py`, the same chart-specific task logic is retained, but the output channel is more tightly constrained:

- the system prompt requests internal reasoning only,
- visible output is constrained to a strict `<answer>...</answer>` form,
- the visible `<think>` handoff is pre-closed before generation,
- only the tagged final answer is extracted and passed to evaluation.

In both manual files, the effective batch size remains 1, no quantization is used, and greedy decoding is preserved throughout.

A.3 Evolution implementation details

Both evolution scripts share the same overall loop:

1. initialize the population from a strong manual seed,

2. request a program mutation,
3. validate the edited program before execution,
4. evaluate the candidate on the local benchmark,
5. score the candidate by the joint objective,
6. update elites and the Pareto frontier,
7. archive logs and repeat.

The mutation operator is a local cached Qwen-based text mutator using Qwen/Qwen3-VL-2B-Instruct. Mutations are proposed as compact XML find/replace edits so that the system evolves the code directly rather than merely changing a user prompt string. The hybrid backend augments this learned mutator with a heuristic mutation bank used when the proposed edit is invalid or otherwise unusable.

To improve robustness and search efficiency, both evolution scripts implement:

- candidate deduplication by code hash,
- candidate validation before runtime evaluation,
- archived candidate modules,
- per-generation summaries,
- explicit elite retention,
- Pareto frontier tracking,
- final population snapshots.

A.4 Search settings

Table 5 lists the main search configurations used in this project.

Table 5: Evolution search settings used in the project.

Run	Samples	Generations	Cand./gen	Population	Elites	Backend
Instruct proxy	8	2	1	4	2	local_qwen
Instruct full	128	4	5	8	3	hybrid
Thinking full	128	2	4	6	2	hybrid

A.5 Supporting artifacts

In addition to the submitted code files, the project includes supporting documentation and run artifacts that were used to verify the final report:

- day-by-day notes:
 - day1_setup_baseline.md
 - day2_manual_instruct.md
 - day3_manual_instruct_speed.md
 - day4_manual_thinking.md
 - day5_evolve_instruct.md
 - day6_evolve_instruct.md
 - day7_thinking_and_bests.md
 - day8_repro_and_summary.md
- full command log:
 - full_pipeline_commands.md
- canonical run directories:
 - evolution_runs/instruct_proxy_day5_canonical
 - evolution_runs/instruct_day6_full

- evolution_runs/thinking_day7_full
- evolution_runs/instruct_final_audit
- evolution_runs/thinking_final_audit
- fresh-process reproduction summary:
 - repro_outputs/summary.md
 - repro_outputs/summary.json

B Additional Search Logs and Failure Cases

B.1 Representative search outcomes

Table 6 restates the most important discovered candidates from the canonical search runs. These are useful for understanding how the search behaved beyond the final submission mapping.

Table 6: Representative candidates discovered during search.

Run	Candidate ID	Accuracy	Avg/query (s)	Main mutation summary
Instruct full best	g00_c005	0.6640625	0.40914330817759037	Stronger global Not Applicable rule
Instruct Pareto speed-leaning	g02_c015	0.65625	0.3535063248127699	Simplified no-prefix answer behavior
Thinking full best	g01_c008	0.578125	0.607107363641262	Tighter line-question guidance on non-line plots

These search logs reinforce the main pattern discussed in the paper: the strongest mutations were conservative and local rather than architecturally dramatic. In particular, the search repeatedly improved rejection behavior for absent chart elements, filtering of legend-like and colorbar-like distractors, and answer formatting discipline.

B.2 Negative search outcomes

Not all search stages produced improvements over their seeds. The Day 5 proxy Instruct evolution run did not outperform the manually engineered seed. In addition, later audit reruns did not surpass the strongest earlier frozen artifacts. These negative outcomes are important for two reasons. First, they show that the evolution loop was not trivially engineered to guarantee monotonic improvements. Second, they justify our decision to select final artifacts based on verified fresh-process results rather than optimistic intermediate observations.

B.3 Failure-case logging

We retained per-query outputs for the final manual and evolved systems in JSON files:

- manual_instruct_day2_final_queries.json
- manual_thinking_day4_final_queries.json
- evolved_instruct_day6_final_queries.json
- evolved_thinking_day7_final_queries.json

These files make it possible to inspect where the systems still fail after optimization. While we do not enumerate individual examples in the main paper, the logged outputs show that the remaining failure modes are concentrated in a familiar set of chart-specific confusions:

- confusing legends with nearby captions or other surrounding text,
- confusing colorbar ranges with axis ranges or heatmap values,
- answering line-related questions too aggressively on non-line plots,
- producing evaluator-incompatible short answers when the visual answer is semantically plausible but formatted incorrectly.

These residual failure patterns are consistent with the overall project narrative. The optimization pipeline substantially reduces output-formatting mistakes and false positives, but it does not eliminate all ambiguity in realistic scientific charts.

B.4 Search logs available for visualization

The archived run directories contain structured files that can be used to produce additional diagnostic visualizations if desired:

- generation summaries:
 - `evolution_runs/instruct_day6_full/generation_summaries.jsonl`
 - `evolution_runs/thinking_day7_full/generation_summaries.jsonl`
- Pareto frontiers:
 - `evolution_runs/instruct_day6_full/pareto_frontier.json`
 - `evolution_runs/thinking_day7_full/pareto_frontier.json`
- population snapshots:
 - `evolution_runs/instruct_day6_full/population.json`
 - `evolution_runs/thinking_day7_full/population.json`

In a longer version of the paper, these logs could be used to visualize score trajectories across generations or to plot the discovered accuracy-latency Pareto frontier. In the current short report, we keep the presentation table-driven and use the appendix primarily to document that these additional search traces were preserved.